

ProblemsOnSets,Dictionary,Heap

le06 · 14,15,16/26



Contenido

Problems On sets
Problems On dictionary
Heap Problems

Problems On sets

```
In [3]:  
s = set(map(int , input().split()))  
print(s)  
print(len(s))
```

```
Out [3]:  
{1, 2, 3, 4, 5}  
5
```

```
In [ ]:  
s = set()  
  
while True:  
    print("-----")  
    print("1. Add")  
    print("2. Remove")  
    print("3. Display")  
    print("4. Exit")  
    print("-----")  
  
    case = int(input("Enter your choice: "))  
  
    match case:  
        case 1:  
            x = int(input("Enter Value to add: "))  
            s.add(x)  
            print("Set:", s)  
  
        case 2:  
            x = int(input("Enter Value to remove: "))  
            s.discard(x)  
            print("Set:", s)  
  
        case 3:  
            print(s if len(s) != 0 else "Set is Empty")  
  
        case 4:  
            break  
  
        case _:  
            print("Invalid choice. Try again.")
```

```
Out [ ]:  
-----  
  
1.add  
2.remove  
3.display  
  
-----  
Set is Empty  
{20}
```

```
In [28]:  
  
# Static set  
numbers = {10, 20, 30, 40, 50}  
  
# Sum of elements  
total = sum(numbers)  
  
# Average of elements  
average = total / len(numbers)  
  
print("Set:", numbers)  
print("Sum:", total)  
print("Average:", average)
```

```
Out [28]:  
Set: {50, 20, 40, 10, 30}  
Sum: 150  
Average: 30.0
```

```
In [29]:  
  
# Static sets  
set1 = {1, 2, 3, 4, 5}  
set2 = {4, 5, 6, 7, 8}  
  
# Union  
union_set = set1.union(set2)  
  
# Intersection  
intersection_set = set1.intersection(set2)  
  
print("Set 1:", set1)  
print("Set 2:", set2)  
print("Union:", union_set)  
print("Intersection:", intersection_set)
```

```
Out [29]:  
Set 1: {1, 2, 3, 4, 5}  
Set 2: {4, 5, 6, 7, 8}  
Union: {1, 2, 3, 4, 5, 6, 7, 8}  
Intersection: {4, 5}
```

```
In [30]:  
  
# Static sets  
A = {1, 2, 3, 4, 5}  
B = {4, 5, 6, 7, 8}  
  
# A - B  
a_minus_b = A - B  
  
# B - A  
b_minus_a = B - A  
  
# Symmetric Difference  
sym_diff = A.symmetric_difference(B)  
  
print("Set A:", A)  
print("Set B:", B)  
print("A - B:", a_minus_b)  
print("B - A:", b_minus_a)  
print("Symmetric Difference:", sym_diff)
```

```
Out [30]:  
Set A: {1, 2, 3, 4, 5}  
Set B: {4, 5, 6, 7, 8}  
A - B: {1, 2, 3}  
B - A: {8, 6, 7}  
Symmetric Difference: {1, 2, 3, 6, 7, 8}
```

```
In [31]:
```

```
# Static set
s = {12, 45, 7, 23, 89, 34}

# Using built-in functions
max_builtin = max(s)
min_builtin = min(s)
range_builtin = max_builtin - min_builtin

print("Using Built-in Functions:")
print("Maximum =", max_builtin)
print("Minimum =", min_builtin)
print("Range =", range_builtin)

# Without using built-in max() and min()
s_list = list(s)
max_val = s_list[0]
min_val = s_list[0]

for num in s_list:
    if num > max_val:
        max_val = num
    if num < min_val:
        min_val = num

range_val = max_val - min_val

print("\nWithout Built-in Functions:")
print("Maximum =", max_val)
print("Minimum =", min_val)
print("Range =", range_val)
```

```
Out [31]:
Using Built-in Functions:
Maximum = 89
Minimum = 7
Range = 82

Without Built-in Functions:
Maximum = 89
Minimum = 7
Range = 82
```

```
In [32]:
# Static sets
A = {1, 2, 3}
B = {1, 2, 3, 4, 5}

# Subset
print("A is subset of B:", A.issubset(B))

# Superset
print("B is superset of A:", B.issuperset(A))

# Proper Subset
print("A is proper subset of B:", A < B)

# Proper Superset
print("B is proper superset of A:", B > A)
```

```
Out [32]:
A is subset of B: True
B is superset of A: True
A is proper subset of B: True
B is proper superset of A: True
```

```
In [33]:
# Static sets
A = {1, 2, 3}
B = {4, 5, 6}

# Check if sets are disjoint
```

```
print("A:", A)
print("B:", B)
print("Are A and B disjoint?", A.isdisjoint(B))
```

```
Out [33]:
A: {1, 2, 3}
B: {4, 5, 6}
Are A and B disjoint? True
```

```
In [34]:
# List with duplicates
lst = [1, 2, 2, 3, 4, 4, 5, 1]

# Using set
unique_list = list(set(lst))

print("Original List:", lst)
print("After Removing Duplicates (Using Set):", unique_list)

# List with duplicates
lst = [1, 2, 2, 3, 4, 4, 5, 1]

unique_list = []

for item in lst:
    if item not in unique_list:
        unique_list.append(item)

print("Original List:", lst)
print("After Removing Duplicates (Without Set):", unique_list)
```

```
Out [34]:
Original List: [1, 2, 2, 3, 4, 4, 5, 1]
After Removing Duplicates (Using Set): [1, 2, 3, 4, 5]
Original List: [1, 2, 2, 3, 4, 4, 5, 1]
After Removing Duplicates (Without Set): [1, 2, 3, 4, 5]
```

```
In [35]:
# List
lst = [1, 2, 3, 4, 5]

# Check uniqueness by comparing lengths
if len(lst) == len(set(lst)):
    print("List contains unique elements")
else:
    print("List contains duplicate elements")
```

```
Out [35]:
List contains unique elements
```

```
In [36]:
# Array of numbers
arr = [12, 18, 25, 35]

unique_prime_factors = set()

for num in arr:
    d = 2
    while num > 1:
        while num % d == 0:
            unique_prime_factors.add(d)
            num //= d
        d += 1

print("Unique Prime Factors in Array:", unique_prime_factors)
```

```
Out [36]:
Unique Prime Factors in Array: {2, 3, 5, 7}
```

Problems On dictionary

```
In [8]:
country = ["india" , "china" , "america"]
capitals = ["delhi" , " aa" , "newyork"]

d = {}
for i in range(3):
    d[country[i]] = capitals[i]

print(d['india'])
```

```
Out [8]:
delhi
```

```
In [19]:
import random as rand

d = {}
names = ["ramu", "raghav", "raja", "bheem"]

for i in range(1,10):
    d[f"student_{i}"] = {
        "name": names[rand.randint(0, len(names) - 1)],
        "score": rand.randint(70, 100)
    }

max_student = None
max_score = -1

for sid, info in d.items():
    if info["score"] > max_score:
        max_score = info["score"]
        max_student = sid

print(d)

print("-----")
print("max student:", max_student, d[max_student])
```

```
Out [19]:
{'student_1': {'name': 'ramu', 'score': 94}, 'student_2': {'name': 'raghav', 'score': 82}, 'student_3': {'name': 'bheem', 'score': 71}, 'student_4': {'name': 'bheem', 'score': 79}, 'student_5': {'name': 'raghav', 'score': 74}, 'student_6': {'name': 'bheem', 'score': 91}, 'student_7': {'name': 'raghav', 'score': 74}, 'student_8': {'name': 'bheem', 'score': 84}, 'student_9': {'name': 'ramu', 'score': 96}}
-----
max student: student_9 {'name': 'ramu', 'score': 96}
```

```
In [ ]:
import random

sample_names = ["Alice", "Bob", "Charlie", "David", "Emma", "Frank", "Grace", "Henry", "Ivy", "Jack"]
sample_jobs = ["Manager", "Developer", "Designer", "Analyst", "HR Specialist", "QA Engineer"]

n = int(input("Enter n (number of employees): "))

d = {}

for i in range(1, n + 1):

    name = random.choice(sample_names)
    job = random.choice(sample_jobs)

    salary = random.randint(40000, 120000)
```

```

d[i] = {
    "name": name,
    "job": job,
    "salary": salary
}

print("\nAll employee details (Automatically Generated):")
for emp_id, info in d.items():
    print(f"ID {emp_id}: {info}")

search_name = input("\nEnter employee name to display details: ")

found = False
for emp_id, info in d.items():
    if info["name"].lower() == search_name.lower():
        print(f"\nEmployee details for '{search_name}':")
        print({emp_id: info})
        found = True
        break

if not found:
    print(f"No employee found with name '{search_name}'.")

```

Out []:

```

All employee details (Automatically Generated):
ID 1: {'name': 'David', 'job': 'Designer', 'salary': 56077}
ID 2: {'name': 'Emma', 'job': 'HR Specialist', 'salary': 74172}
ID 3: {'name': 'Ivy', 'job': 'Developer', 'salary': 53904}
ID 4: {'name': 'Frank', 'job': 'QA Engineer', 'salary': 79555}
ID 5: {'name': 'Frank', 'job': 'Developer', 'salary': 87611}
ID 6: {'name': 'Bob', 'job': 'Manager', 'salary': 98063}
ID 7: {'name': 'Henry', 'job': 'Developer', 'salary': 44911}
ID 8: {'name': 'Ivy', 'job': 'Analyst', 'salary': 58858}
ID 9: {'name': 'Charlie', 'job': 'HR Specialist', 'salary': 92298}
ID 10: {'name': 'Grace', 'job': 'Analyst', 'salary': 60871}
ID 11: {'name': 'Ivy', 'job': 'Designer', 'salary': 40421}
ID 12: {'name': 'Emma', 'job': 'Manager', 'salary': 54830}
ID 13: {'name': 'Jack', 'job': 'Designer', 'salary': 83263}
ID 14: {'name': 'Charlie', 'job': 'Developer', 'salary': 104005}
ID 15: {'name': 'Jack', 'job': 'QA Engineer', 'salary': 50166}
ID 16: {'name': 'Jack', 'job': 'HR Specialist', 'salary': 53308}
ID 17: {'name': 'Ivy', 'job': 'Analyst', 'salary': 76391}
ID 18: {'name': 'Alice', 'job': 'Analyst', 'salary': 111543}
ID 19: {'name': 'Grace', 'job': 'Designer', 'salary': 74262}
ID 20: {'name': 'David', 'job': 'HR Specialist', 'salary': 54528}
ID 21: {'name': 'Bob', 'job': 'QA Engineer', 'salary': 105916}
ID 22: {'name': 'Bob', 'job': 'Developer', 'salary': 97955}
ID 23: {'name': 'Henry', 'job': 'Analyst', 'salary': 97705}
ID 24: {'name': 'Ivy', 'job': 'Analyst', 'salary': 97345}
ID 25: {'name': 'David', 'job': 'HR Specialist', 'salary': 52063}
ID 26: {'name': 'Charlie', 'job': 'QA Engineer', 'salary': 83523}
ID 27: {'name': 'Henry', 'job': 'Designer', 'salary': 61467}
ID 28: {'name': 'Alice', 'job': 'Designer', 'salary': 45864}
ID 29: {'name': 'Emma', 'job': 'QA Engineer', 'salary': 91786}
ID 30: {'name': 'Bob', 'job': 'Manager', 'salary': 109599}
ID 31: {'name': 'Henry', 'job': 'HR Specialist', 'salary': 73692}
ID 32: {'name': 'David', 'job': 'Manager', 'salary': 62520}
ID 33: {'name': 'Henry', 'job': 'Analyst', 'salary': 73492}
ID 34: {'name': 'Ivy', 'job': 'Designer', 'salary': 109878}
ID 35: {'name': 'Charlie', 'job': 'QA Engineer', 'salary': 73199}
ID 36: {'name': 'Henry', 'job': 'QA Engineer', 'salary': 65489}
ID 37: {'name': 'Alice', 'job': 'QA Engineer', 'salary': 113355}
ID 38: {'name': 'Henry', 'job': 'Manager', 'salary': 62549}
ID 39: {'name': 'Alice', 'job': 'Manager', 'salary': 45430}
ID 40: {'name': 'Alice', 'job': 'Developer', 'salary': 85415}
ID 41: {'name': 'Jack', 'job': 'HR Specialist', 'salary': 102567}
ID 42: {'name': 'Ivy', 'job': 'Developer', 'salary': 82543}
ID 43: {'name': 'Ivy', 'job': 'QA Engineer', 'salary': 81506}
ID 44: {'name': 'Frank', 'job': 'Analyst', 'salary': 71029}
ID 45: {'name': 'Bob', 'job': 'Manager', 'salary': 87624}
ID 46: {'name': 'Jack', 'job': 'Developer', 'salary': 40995}
ID 47: {'name': 'Emma', 'job': 'QA Engineer', 'salary': 111074}
ID 48: {'name': 'Henry', 'job': 'Manager', 'salary': 49305}
ID 49: {'name': 'Ivy', 'job': 'Manager', 'salary': 108639}

```

```

ID 50: {'name': 'Ivy', 'job': 'Analyst', 'salary': 93723}
ID 51: {'name': 'Grace', 'job': 'HR Specialist', 'salary': 109276}
ID 52: {'name': 'Ivy', 'job': 'Developer', 'salary': 42141}
ID 53: {'name': 'Grace', 'job': 'Manager', 'salary': 58182}
ID 54: {'name': 'Emma', 'job': 'Manager', 'salary': 41955}
ID 55: {'name': 'Emma', 'job': 'QA Engineer', 'salary': 55417}
ID 56: {'name': 'David', 'job': 'HR Specialist', 'salary': 44035}
ID 57: {'name': 'Ivy', 'job': 'Designer', 'salary': 50024}
ID 58: {'name': 'Alice', 'job': 'Analyst', 'salary': 113048}
ID 59: {'name': 'Alice', 'job': 'HR Specialist', 'salary': 58981}
ID 60: {'name': 'Grace', 'job': 'Designer', 'salary': 92420}
ID 61: {'name': 'Emma', 'job': 'Analyst', 'salary': 118555}
ID 62: {'name': 'Ivy', 'job': 'Developer', 'salary': 100784}
ID 63: {'name': 'Jack', 'job': 'HR Specialist', 'salary': 98382}
ID 64: {'name': 'Alice', 'job': 'Developer', 'salary': 51986}
ID 65: {'name': 'Grace', 'job': 'Developer', 'salary': 69018}
ID 66: {'name': 'Grace', 'job': 'QA Engineer', 'salary': 75550}
ID 67: {'name': 'Emma', 'job': 'Manager', 'salary': 48928}
ID 68: {'name': 'Frank', 'job': 'QA Engineer', 'salary': 116108}
ID 69: {'name': 'Bob', 'job': 'QA Engineer', 'salary': 81075}
ID 70: {'name': 'David', 'job': 'HR Specialist', 'salary': 73174}
ID 71: {'name': 'Alice', 'job': 'Developer', 'salary': 90627}
ID 72: {'name': 'Jack', 'job': 'Developer', 'salary': 59705}
ID 73: {'name': 'Ivy', 'job': 'Manager', 'salary': 99312}
ID 74: {'name': 'Ivy', 'job': 'HR Specialist', 'salary': 41745}
ID 75: {'name': 'David', 'job': 'Designer', 'salary': 54660}
ID 76: {'name': 'Ivy', 'job': 'Manager', 'salary': 103493}
ID 77: {'name': 'Charlie', 'job': 'QA Engineer', 'salary': 113580}
ID 78: {'name': 'Bob', 'job': 'Manager', 'salary': 92053}
ID 79: {'name': 'Emma', 'job': 'QA Engineer', 'salary': 118503}
ID 80: {'name': 'Frank', 'job': 'Developer', 'salary': 117828}
ID 81: {'name': 'Grace', 'job': 'Manager', 'salary': 42986}
ID 82: {'name': 'Grace', 'job': 'Manager', 'salary': 108806}
ID 83: {'name': 'Emma', 'job': 'Analyst', 'salary': 81049}
ID 84: {'name': 'Jack', 'job': 'Designer', 'salary': 109376}
ID 85: {'name': 'Frank', 'job': 'Designer', 'salary': 110609}
ID 86: {'name': 'Emma', 'job': 'Manager', 'salary': 119784}
ID 87: {'name': 'Ivy', 'job': 'Analyst', 'salary': 60158}
ID 88: {'name': 'Jack', 'job': 'QA Engineer', 'salary': 97098}
ID 89: {'name': 'Henry', 'job': 'Designer', 'salary': 47232}
ID 90: {'name': 'Jack', 'job': 'Manager', 'salary': 114016}
ID 91: {'name': 'Ivy', 'job': 'Manager', 'salary': 67862}
ID 92: {'name': 'Grace', 'job': 'Designer', 'salary': 90580}
ID 93: {'name': 'David', 'job': 'HR Specialist', 'salary': 77003}
ID 94: {'name': 'Frank', 'job': 'Analyst', 'salary': 108402}
ID 95: {'name': 'Bob', 'job': 'QA Engineer', 'salary': 58911}
ID 96: {'name': 'Alice', 'job': 'Analyst', 'salary': 106425}
ID 97: {'name': 'Ivy', 'job': 'Manager', 'salary': 66896}
ID 98: {'name': 'Bob', 'job': 'HR Specialist', 'salary': 70931}
ID 99: {'name': 'Bob', 'job': 'HR Specialist', 'salary': 97434}
ID 100: {'name': 'Alice', 'job': 'QA Engineer', 'salary': 97304}

```

Employee details for 'bob':

```
{6: {'name': 'Bob', 'job': 'Manager', 'salary': 98063}}
```

In [21]:

```

d1 = {"India":140 , "china":130 , "Srilanka":12}
d2 = {"USA":40 , "france":10 , "Argentica":6}
d3 = {"UK":26 , "France":14}
d4 = {"SA":14 , "Zimbambe":3}
d5 = {}

```

```

for d in (d1 , d2 , d3 , d4):
    d5.update(d)
print(d5)

```

Out [21]:

```
{'India': 140, 'china': 130, 'Srilanka': 12, 'USA': 40, 'france': 10, 'Argentica': 6, 'UK': 26, 'France': 14, 'SA': 14, 'Zimbambe': 3}
```

In [2]:

```

d = { "India" : 30 , "China":40 , "Russia":30 , "Europe":80}
print(f"Original dictionary {d}")
d = dict(sorted(d.items()))

```

```
print("Sorted based one Country",d)
d = dict(sorted(d.items() , key=lambda x:x[1]))
print("Sorted Based On poulation",d)
```

Out [2]:

```
Original dictionary {'India': 30, 'China': 40, 'Russia': 30, 'Europe': 80}
Sorted based one Country {'China': 40, 'Europe': 80, 'India': 30, 'Russia': 30}
Sorted Based On poulation {'India': 30, 'Russia': 30, 'China': 40, 'Europe': 80}
```

In [10]:

```
d = {}
print(type(d))
print(len(d))

while True:
    print("-----\n1.add a pair \n2.sort by name \n3.sort by price \n4.sort by quantity \n5.sort

c = int(input("Enter The Choice"))

match c:
    case 1:
        key = len(d)+1

        name = input("Enter Name of the Product")
        price = int(input("Enter price of the Product"))
        qty = int(input("Enter quantity of the Product"))

        rec = {"name":name , "price":price , "qty":qty}

        d[key] = rec

        print(d)

    case 2:
        print("Sort By name Assending Order", dict(sorted(d.items(), key=lambda x: x[1]["name"])))
        print("Sort By name desending Order", dict(sorted(d.items(), key=lambda x: x[1]["name"], reverse=True)))

    case 3:
        print("Sort By price Assending Order", dict(sorted(d.items(), key=lambda x: x[1]["price"])))
        print("Sort By price desending Order", dict(sorted(d.items(), key=lambda x: x[1]["price"], reverse=True)))

    case 4:
        print("Sort By qty Assending Order", dict(sorted(d.items(), key=lambda x: x[1]["qty"])))
        print("Sort By qty desending Order", dict(sorted(d.items(), key=lambda x: x[1]["qty"], reverse=True)))

    case 5:
        print("Sort By (name , price) Assending Order", dict(sorted(d.items(), key=lambda x: (x[1]["name"], x[1]['p
        print("Sort By (name ,price) desending Order", dict(sorted(d.items(), key=lambda x: (x[1]["name"], x[1]['pr

    case 6:
        print("Sort By (name , qty) Assending Order", dict(sorted(d.items(), key=lambda x: (x[1]["name"], x[1]['qty
        print("Sort By (name ,qty) desending Order", dict(sorted(d.items(), key=lambda x: (x[1]["name"], x[1]['qty'

    case 7:
        print("Sort By (price , qty) Assending Order", dict(sorted(d.items(), key=lambda x: (x[1]["price"], x[1]['q
        print("Sort By (price ,qty) desending Order", dict(sorted(d.items(), key=lambda x: (x[1]["price"], x[1]['qt

    case 8:
        print("Sort By (price , qty , name) Assending Order", dict(sorted(d.items(), key=lambda x: (x[1]["price"],
        print("Sort By (price ,qty , name) desending Order", dict(sorted(d.items(), key=lambda x: (x[1]["price"], x

    case 9:
        print(f"Original Dictionary{d}")

    case _ :
        break
```



```

Out [10]:
<class 'dict'>
0
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display
{1: {'name': 'paper', 'price': 160, 'qty': 20}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display
{1: {'name': 'paper', 'price': 160, 'qty': 20}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display
{1: {'name': 'paper', 'price': 160, 'qty': 20}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 3: {'name': 'tv', 'price': 680000, 'qty': 32}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display
Sort By name Assending Order {2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 1: {'name': 'paper', 'price': 160, 'qty': 20}, 3: {'name': 'tv', 'price': 680000, 'qty': 32}}
Sort By name desending Order {3: {'name': 'tv', 'price': 680000, 'qty': 32}, 1: {'name': 'paper', 'price': 160, 'qty': 20}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display
Sort By price Assending Order {1: {'name': 'paper', 'price': 160, 'qty': 20}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 3: {'name': 'tv', 'price': 680000, 'qty': 32}}
Sort By price desending Order {3: {'name': 'tv', 'price': 680000, 'qty': 32}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 1: {'name': 'paper', 'price': 160, 'qty': 20}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display

```

```

Sort By qty Assending Order {1: {'name': 'paper', 'price': 160, 'qty': 20}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 3: {'name': 'tv', 'price': 680000, 'qty': 32}}
Sort By qty desending Order {3: {'name': 'tv', 'price': 680000, 'qty': 32}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 1: {'name': 'paper', 'price': 160, 'qty': 20}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display
Sort By (name , price) Assending Order {2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 1: {'name': 'paper', 'price': 160, 'qty': 20}, 3: {'name': 'tv', 'price': 680000, 'qty': 32}}
Sort By (name ,price) desending Order {3: {'name': 'tv', 'price': 680000, 'qty': 32}, 1: {'name': 'paper', 'price': 160, 'qty': 20}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display
Sort By (name , qty) Assending Order {2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 1: {'name': 'paper', 'price': 160, 'qty': 20}, 3: {'name': 'tv', 'price': 680000, 'qty': 32}}
Sort By (name ,qty) desending Order {3: {'name': 'tv', 'price': 680000, 'qty': 32}, 1: {'name': 'paper', 'price': 160, 'qty': 20}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display
Sort By (price , qty) Assending Order {1: {'name': 'paper', 'price': 160, 'qty': 20}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 3: {'name': 'tv', 'price': 680000, 'qty': 32}}
Sort By (price ,qty) desending Order {3: {'name': 'tv', 'price': 680000, 'qty': 32}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 1: {'name': 'paper', 'price': 160, 'qty': 20}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display
Sort By (price , qty , name) Assending Order {1: {'name': 'paper', 'price': 160, 'qty': 20}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 3: {'name': 'tv', 'price': 680000, 'qty': 32}}
Sort By (price ,qty , name) desending Order {3: {'name': 'tv', 'price': 680000, 'qty': 32}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 1: {'name': 'paper', 'price': 160, 'qty': 20}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price
6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display
Original Dictionary{1: {'name': 'paper', 'price': 160, 'qty': 20}, 2: {'name': 'laptop', 'price': 120000, 'qty': 24}, 3: {'name': 'tv', 'price': 680000, 'qty': 32}}
-----
1.add a pair
2.sort by name
3.sort by price
4.sort by quantity
5.sort by name , price

```

```

6.sort by name , qunatity
7.sort by price , qunatity
8.sort by price quantity name
9.display

```

```

In [24]:
from collections import Counter
l = [ 10 , 5 , 10 , 15 , 10 , 5]

d = dict(Counter(l))
print(type(d) , "\t" , d)

print("Element : Frequency")
for x in l:
    print(f"{x}          : {d[x]}")

```

```

Out [24]:
<class 'dict'>    {10: 3, 5: 2, 15: 1}
Element : Frequency
10          : 3
5           : 2
10          : 3
15          : 1
10          : 3
5           : 2

```

```

In [48]:
from collections import Counter
l = [10, 5, 10, 15, 10, 5]

d = dict(Counter(l))
max_frequency = max(d.values())

for element, frequency in d.items():
    if frequency == max_frequency:
        print(f"Element: {element}, Frequency: {frequency}")
        break

```

```

Out [48]:
Element: 10, Frequency: 3

```

```

In [57]:
from collections import Counter
l = [10, 5, 10, 15, 10, 5 , 5]

d = dict(Counter(l))
max_frequency = max(d.values())

for element, frequency in d.items():
    if frequency == max_frequency:
        print(f"Element: {element}, Frequency: {frequency}")

```

```

Out [57]:
Element: 10, Frequency: 3
Element: 5, Frequency: 3

```

```

In [64]:
from collections import Counter
l = [10, 5, 10, 15, 10, 5 , 5]

d = dict(Counter(l))
max_frequency = max(d.values())

a = []
for element, frequency in d.items():

```

```
    if frequency == max_frequency:
        print(f"Element: {element}, Frequency: {frequency}")
        a.append(element)
print("Max element",max(a))
```

```
Out [64]:
Element: 10, Frequency: 3
Element: 5, Frequency: 3
Max element 10
```

```
In [65]:
from collections import Counter
l = [10, 5, 10, 15, 10, 5, 5]

d = dict(Counter(l))
max_frequency = max(d.values())

a = []
for element, frequency in d.items():
    if frequency == max_frequency:
        print(f"Element: {element}, Frequency: {frequency}")
        a.append(element)
print("Minimum element",min(a))
```

```
Out [65]:
Element: 10, Frequency: 3
Element: 5, Frequency: 3
Minimum element 5
```

```
In [ ]:
from collections import Counter
l = [1,1,1,2,2,2,4,4,4,6,6,7,8]

d = dict(Counter(l))
max_frequency = max(d.values())
print(max_frequency)

for element, frequency in d.items():
    if frequency == max_frequency:
        print(f"Element: {element}, Frequency: {frequency}")
```

```
Out [ ]:
3
Element: 1, Frequency: 3
Element: 2, Frequency: 3
Element: 4, Frequency: 3
Element: 6, Frequency: 3
```

```
In [ ]:
from collections import Counter
l = [1,1,1,2,2,2,4,4,4,6,6,7,8]

d = dict(Counter(l))
min_freq = min(d.values())
print(min_freq)

for element, frequency in d.items():
    if frequency == min_freq:
        print(f"Element: {element}, Frequency: {frequency}")
```

```
Out [ ]:
```

```
1
Element: 7, Frequency: 1
Element: 8, Frequency: 1
```

```
In [ ]:
from collections import Counter
l = [1,1,1,2,2,2,4,4,4,6,6,7,8]

d = dict(Counter(l))
min_freq = min(d.values())
print(min_freq)

a = []
for element, frequency in d.items():
    if frequency == min_freq:
        print(f"Element: {element}, Frequency: {frequency}")
        a.append(element)
print(max(a))
```

```
Out [ ]:
1
Element: 7, Frequency: 1
Element: 8, Frequency: 1
8
```

```
In [ ]:
from collections import Counter
l = [1,1,1,2,2,2,4,4,4,6,6,7,8]

d = dict(Counter(l))
min_freq = min(d.values())
print(min_freq)

a = []
for element, frequency in d.items():
    if frequency == min_freq:
        print(f"Element: {element}, Frequency: {frequency}")
        a.append(element)
print("count of lowest element frequencys", len(a))
```

```
Out [ ]:
1
Element: 7, Frequency: 1
Element: 8, Frequency: 1
count of lowest element frequencys 2
```

```
In [72]:
from collections import Counter
l = [1,1,1,2,2,2,4,4,4,6,6,7,8]

d = dict(Counter(l))
k = 3
print(k)

a = []
for element, frequency in d.items():
    if frequency == k:
        print(f"Element: {element}, Frequency: {frequency}")
```

```
Out [72]:
3
Element: 1, Frequency: 3
Element: 2, Frequency: 3
```

```
Element: 4, Frequency: 3
Element: 6, Frequency: 3
```

```
In [73]:
from collections import Counter
l = [1,1,1,2,2,2,4,4,4,6,6,6,7,8]

d = dict(Counter(l))
k = 3
print(k)

a = []
for element, frequency in d.items():
    if frequency == k:
        print(f"Element: {element}, Frequency: {frequency}")
        a.append(element)
print(len(a))
```

```
Out [73]:
3
Element: 1, Frequency: 3
Element: 2, Frequency: 3
Element: 4, Frequency: 3
Element: 6, Frequency: 3
4
```

```
In [75]:
l = [1,1,1,2,3,4,3,3,5,6,5,4,4]
print(l)
l=list(set(l))
print("After Removing duplicates",l)
print("Total no of unique elements",len(l))
```

```
Out [75]:
[1, 1, 1, 2, 3, 4, 3, 3, 5, 6, 5, 4, 4]
After Removing duplicates [1, 2, 3, 4, 5, 6]
Total no of unique elements 6
```

```
In [78]:

import collections
print(collections.__all__)
```

```
Out [78]:
['ChainMap', 'Counter', 'OrderedDict', 'UserDict', 'UserList', 'UserString', 'defaultdict', 'deque', 'namedtuple']
```

```
In [96]:
from collections import *
a = [1,2,2,2,4,5,3,3,3,2,4]
a1=['1','2']

print(ChainMap(a))
print(Counter(a))
print(OrderedDict(Counter(a)))
print(UserDict(Counter(a)))
print(UserList(a))
```

```
Out [96]:
ChainMap([1, 2, 2, 2, 4, 5, 3, 3, 3, 2, 4])
Counter({2: 4, 3: 3, 4: 2, 1: 1, 5: 1})
OrderedDict({1: 1, 2: 4, 4: 2, 5: 1, 3: 3})
{1: 1, 2: 4, 4: 2, 5: 1, 3: 3}
[1, 2, 2, 2, 4, 5, 3, 3, 3, 2, 4]
```

Heap Problems

```
In [1]:  
import heapq as h
```

```
In [7]:  
l = []  
h.heapify(l)  
while True:  
    print("1.Insert")  
    print("2.Delete")  
    print("3.Find minimum")  
    print("4.exit")  
    c = int(input("Enter Choice"))  
  
    match c:  
        case 1:  
            x = int(input('Enter Element to insert'))  
            h.heappush(l,x)  
  
            print(l)  
        case 2:  
  
            h.heappop(l)  
  
            print(l)  
        case 3:  
            print(l[0])  
        case 4:  
            break
```

```
Out [7]:  
1.Insert  
2.Delete  
3.Find minimum  
4.exit  
[25]  
1.Insert  
2.Delete  
3.Find minimum  
4.exit  
[20, 25]  
1.Insert  
2.Delete  
3.Find minimum  
4.exit  
[15, 25, 20]  
1.Insert  
2.Delete  
3.Find minimum  
4.exit  
[15, 25, 20, 30]  
1.Insert  
2.Delete  
3.Find minimum  
4.exit  
[15, 25, 20, 30, 40]  
1.Insert  
2.Delete  
3.Find minimum  
4.exit  
[15, 25, 16, 30, 40, 20]  
1.Insert  
2.Delete  
3.Find minimum  
4.exit  
[16, 25, 20, 30, 40]
```

```

1.Insert
2.Delete
3.Find minimum
4.exit
16
1.Insert
2.Delete
3.Find minimum
4.exit

```

```

In [ ]:
l = []
h._heapify_max(l)
while True:
    print("1.Insert")
    print("2.Delete")
    print("3.Find minimum")
    print("4.exit")
    c = int(input("Enter Choice"))

    match c:
        case 1:
            x = int(input('Enter Element to insert'))
            h.heappush(l,x)

            print(l)
        case 2:

            h._heappop_max(l)

            print(l)
        case 3:
            print(l[0])
        case 4:
            break

```

```

In [13]:
l = [1,5,2,3,5,7,9,2]
print(l)
h.heapify(l)

print(l)
print(l[0])

```

```

Out [13]:
[1, 5, 2, 3, 5, 7, 9, 2]
[1, 2, 2, 3, 5, 7, 9, 5]
1

```

```

In [15]:
l = [1,5,2,3,5,7,9,2]
print(l)
h._heapify_max(l)

print(l)
print(l[0])

```

```

Out [15]:
[1, 5, 2, 3, 5, 7, 9, 2]
[9, 5, 7, 3, 5, 1, 2, 2]
9

```

```

In [17]:
l = [1,5,2,3,5,7,9,2]
print(l)

```



```

h._heapify_max(l)

k = 3

x=0
for i in range(k):
    x=h.heappop_max(l)
print(x)

```

```

Out [17]:
[1, 5, 2, 3, 5, 7, 9, 2]
5

```

```

In [25]:
l = [1,5,2,3,5,7,9,2]

#l = [-x for x in m]

print(l)
h.heapify(l)

sorted_list = []
for i in range(len(l)):
    sorted_list.append(h.heappop(l))

print(sorted_list)

```

```

Out [25]:
[1, 5, 2, 3, 5, 7, 9, 2]
[1, 2, 2, 3, 5, 5, 7, 9]

```

```

In [27]:
m = [1,5,2,3,5,7,9,2]

l = [-x for x in m]

print(l)
h.heapify(l)

sorted_list = []
for i in range(len(l)):
    sorted_list.append(h.heappop(l))

m = [-x for x in sorted_list]
print(m)

```

```

Out [27]:
[-1, -5, -2, -3, -5, -7, -9, -2]
[9, 7, 5, 5, 3, 2, 2, 1]

```

```

In [ ]:
l = [1,5,2,3,5,7,9,2]
print(l)
h._heapify_max(l)

sorted_list = []
for i in range(len(l)):
    sorted_list.append(h.heappop_max(l))

print(sorted_list)

```

```

Out [ ]:
[1, 5, 2, 3, 5, 7, 9, 2]
[9, 7, 5, 5, 3, 2, 2, 1]

```